COMSATS University Islamabad

Attock Campus

Department Of Computer Science

Course	***Information Security-Lab***
Instructor	***Mam Ambreen***
Assignment No.	***04***

Student Details

Registration No.	***Name***
Fa24-bse-039	***Muhammad Taha Niaz***

TASK: 1

Simple RSA Implementation

Code:

```python
import random
# --- Helper Functions ---
def gcd(a, b):  1 usage
    """Euclidean algorithm to find the greatest common divisor."""
    while b:
        a, b = b, a % b
    return a

def multiplicative_inverse(e, phi):  1 usage
    """Extended Euclidean Algorithm to find the modular multiplicative inverse."""
    d = 0
    x1, x2 = 0, 1
    y1, y2 = 1, 0
    temp_phi = phi
    while e > 0:
        temp1 = temp_phi // e
        temp2 = temp_phi - temp1 * e
        temp_phi = e
        e = temp2
        x = x2 - temp1 * x1
        y = y2 - temp1 * y1
        x2, x1 = x1, x
        y2, y1 = y1, y
    if temp_phi == 1:
        return y2 % phi

# --- Core RSA Functions ---
def generate_keypair(p, q):  1 usage
    # n = p * q
    n = p * q
    # Phi is Euler's totient function
    phi = (p - 1) * (q - 1)
```

```
    # Choose an integer e such that e and phi(n) are coprime
    e = 65537  # Common choice for e
    if gcd(e, phi) != 1:
        e = 3  # Fallback for very small primes
    # Use Extended Euclidean Algorithm to generate the private key
    d = multiplicative_inverse(e, phi)
    # Return Public Key (e, n) and Private Key (d, n)
    return ((e, n), (d, n))

def encrypt(public_key, plaintext):  1 usage
    e, n = public_key
    # Convert characters to numbers and compute: c = m^e mod n
    cipher = [pow(ord(char), e, n) for char in plaintext]
    return cipher

def decrypt(private_key, ciphertext):  1 usage
    d, n = private_key
    # Compute: m = c^d mod n and convert back to characters
    plain = [chr(pow(char, d, n)) for char in ciphertext]
    return ''.join(plain)

# --- Testing ---
# 1. Choose two small prime numbers
p = 61
q = 53
print(f"Generating keys with p={p} and q={q}...")
public, private = generate_keypair(p, q)
print(f"Public Key: {public}")
print(f"Private Key: {private}")
```

```
# 2. Test with a name
message = "M.Taha Niaz"
print(f"\nOriginal Message: {message}")

# 3. Encrypt
encrypted_msg = encrypt(public, message)
print(f"Encrypted (Ciphertext): {encrypted_msg}")

# 4. Decrypt
decrypted_msg = decrypt(private, encrypted_msg)
print(f"Decrypted Message: {decrypted_msg}")
```

Output:

```
C:\Users\HP\IdeaProjects\PythonProject\PythonProject\PythonProject\PythonProject\PythonProje
Generating keys with p=61 and q=53...
Public Key: (65537, 3233)
Private Key: (2753, 3233)

Original Message: M.Taha Niaz
Encrypted (Ciphertext): [3123, 2825, 2159, 1632, 2170, 1632, 1992, 3165, 3179, 1632, 1830]
Decrypted Message: M.Taha Niaz

Process finished with exit code 0
```

TASK: 2

Digital Signature

Code:

```
port hashlib

--- FUNCTIONS FROM TASK 1 (Must be present) ---
f gcd(a, b):  1 usage
  while b:
      a, b = b, a % b
  return a

f multiplicative_inverse(e, phi):  1 usage
  d, x1, x2, y1, y2 = 0, 0, 1, 1, 0
  temp_phi = phi
  while e > 0:
      temp1, temp2 = divmod(temp_phi, e)
      temp_phi, e = e, temp2
      x, y = x2 - temp1 * x1, y2 - temp1 * y1
      x2, x1, y2, y1 = x1, x, y1, y
  return y2 % phi if temp_phi == 1 else None

f generate_keypair(p, q):  1 usage
  n = p * q
  phi = (p - 1) * (q - 1)
  e = 65537
  if gcd(e, phi) != 1: e = 3
  d = multiplicative_inverse(e, phi)
  return ((e, n), (d, n))

--- FUNCTIONS FOR TASK 2 ---

f sign_hash(private_key, message_hash):  1 usage
  d, n = private_key
  hash_int = int(message_hash, 16) % n
```

```python
    return pow(hash_int, d, n)

def verify_signature(public_key, message_hash, signature):  2 usages
    e, n = public_key
    hash_int = int(message_hash, 16) % n
    return pow(signature, e, n) == hash_int

# --- EXECUTION WORKFLOW ---
# 1. Setup Keys
p, q = 101, 103
public_key, private_key = generate_keypair(p, q)

# 2. Original Message
original_message = "Transfer $100 to Alice"
msg_hash = hashlib.sha256(original_message.encode()).hexdigest()
signature = sign_hash(private_key, msg_hash)

print(f"Message: {original_message}")
print(f"Verification: {'SUCCESS' if verify_signature(public_key, msg_hash, signature) else 'FAILED'}")

# 3. Tampered Message
tampered_message = "Transfer $1000 to Alice"
tampered_hash = hashlib.sha256(tampered_message.encode()).hexdigest()

print(f"\nTampered: {tampered_message}")
print(f"Verification: {'SUCCESS' if verify_signature(public_key, tampered_hash, signature) else 'FAILED'}")
```

Output:

```
C:\Users\HP\IdeaProjects\PythonProject\PythonProject\
Message: Transfer $100 to Alice
Verification: SUCCESS

Tampered: Transfer $1000 to Alice
Verification: FAILED

Process finished with exit code 0
```